

LAPORAN 4

Analisis Kompleksitas Algoritma

Disusun untuk memenuhi tugas laporan



DISUSUN OLEH :

AMARA IMANIKA
NIM 2025573010042

Kelas	: TI 1C
Mata Kuliah	: Algoritma dan Struktur Data
Jurusan	: Teknologi Informasi dan Komputer
Prodi	: Teknik Informatika
Dosen	: Muhammad Reza Zulman, S.ST., M.Sc

PROGRAM STUDI INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER
POLITEKNIK NEGERI LHOKSEUMAWE
2025 - 2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa atas selesainya laporan praktikum ini yang berjudul "Analisis Kompleksitas Algoritma". Laporan ini disusun sebagai bagian dari pemenuhan tugas mata kuliah Struktur Data dan Algoritma untuk mendokumentasikan pemahaman mengenai efisiensi sebuah algoritma.

Penulis mengucapkan terima kasih kepada dosen pengampu yang telah memberikan bimbingan serta materi pada Modul 5 ini. Penulis menyadari bahwa laporan ini masih memerlukan penyempurnaan, oleh karena itu saran dan kritik yang membangun sangat diharapkan demi perbaikan di masa mendatang.

Lhokseumawe, 28 April 2026

Penulis

Amara Imanika

DAFTAR ISI

DAFTAR ISI	3
BAB I PENDAHULUAN	4
1.1. Latar Belakang	4
1.2. Tujuan Penulisan	4
1.3. Manfaat Penulisan	5
BAB II PEMBAHASAN	6
2.1. Alur kerja Pemograman Analisis Kompleksitas Algoritma	6
2.2. Latihan dan Tugas Pratikum	17
2.3. Melakukan Commit dan Push	24
BAB III PENUTUP	25
3.1. Kesimpulan	25
3.2. Saran	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman, sebuah masalah seringkali dapat diselesaikan dengan berbagai algoritma yang berbeda. Namun, setiap algoritma memiliki karakteristik yang berbeda dalam penggunaan sumber daya komputer, terutama waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*). Analisis kompleksitas menjadi sangat penting karena memungkinkan pengembang untuk memprediksi performa suatu program sebelum diimplementasikan secara luas, terutama saat menangani volume data yang besar. Tanpa analisis yang tepat, sistem yang dibangun berisiko mengalami kegagalan performa ketika beban data meningkat. Oleh karena itu, pemahaman mengenai notasi Big-O dan perhitungan langkah operasional sangat diperlukan oleh setiap mahasiswa informatika.

1.2 Tujuan Penulisan

Berdasarkan materi yang terdapat dalam Modul 5, tujuan dari pelaksanaan praktikum dan penyusunan laporan ini adalah agar mahasiswa mampu:

1. **Memahami Konsep Efisiensi Algoritma:** Mahasiswa dapat membedakan antara algoritma yang efisien dan tidak efisien berdasarkan penggunaan sumber daya (waktu dan ruang).
2. **Menguasai Notasi Big-O:** Mahasiswa mampu menggunakan notasi Big-O (O) untuk mengklasifikasikan algoritma ke dalam berbagai tingkat kompleksitas (seperti konstan, linear, kuadratik, atau logaritmik).
3. **Melakukan Analisis Worst-Case:** Mahasiswa dapat menghitung batas atas (*upper bound*) dari suatu fungsi atau perulangan untuk memprediksi performa maksimal sebuah program saat menangani data dalam jumlah besar.
4. **Menghitung Langkah Operasional:** Mahasiswa mampu menguraikan setiap baris kode program dan menghitung total langkah eksekusi secara matematis untuk mendapatkan fungsi kompleksitas $f(n)$.
5. **Menerapkan Optimasi Kode:** Mahasiswa dapat mengambil keputusan teknis dalam memodifikasi kode program agar memiliki kompleksitas yang lebih rendah guna meningkatkan performa aplikasi.

1.3 Manfaat Penulisan

Manfaat dari penyusunan laporan dan pelaksanaan praktikum ini adalah:

1. **Bagi Penulis:** Meningkatkan kemampuan dalam mengevaluasi algoritma mana yang paling efisien untuk memecahkan masalah tertentu.
2. **Bagi Pembaca:** Memberikan referensi mengenai cara menghitung kompleksitas waktu secara teoritis dan praktis melalui kode program.
3. **Secara Teknis:** Membantu dalam mengoptimalkan kode program agar dapat berjalan lebih cepat dan menggunakan memori yang lebih sedikit.

BAB II

PEMBAHASAN

2.1. Alur kerja Pemograman Analisis Kompleksitas Algoritma

Langkah 1 | Buka Git Bash. Kemudian navigation ke root folder repository kamu.

```
cd nama – repository - kamu
```

Langkah 2 | Membuat folder pratikum-5

```
mkdir praktikum-5  
cd praktikum-5
```

Langkah 3 | Inisialisasi npm project.

```
npm init -y
```

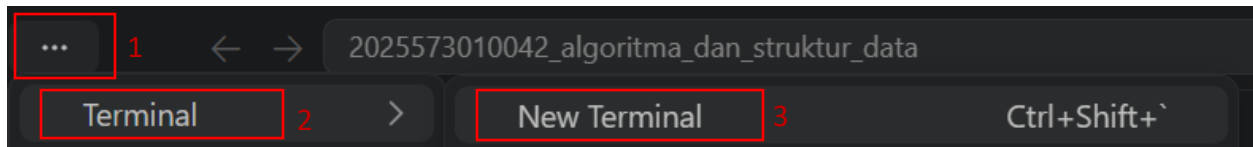
Langkah 4 |

Pastikan berada dalam modul-5, kemudian buat file baru dengan nama 01-intro-kompleksitas.js

Kemudian di halaman file **01-intro-kompleksitas.js** ketikkan kode dibawah ini :

```
JS 01-intro-kompleksitas.js X
pratikum-5 > JS 01-intro-kompleksitas.js > ...
1  function ukurWaktu(label, fn) {
2      const mulai = Date.now();
3      fn();
4      const selesai = Date.now();
5      console.log(`${label}: ${selesai - mulai} ms`);
6  }
7  const N = 100_000; // ukuran data: 100 ribu
8
9  function jumlahkanLinear(n) {
10     let total = 0;
11     for (let i = 1; i <= n; i++) total += i;
12     return total;
13 }
14
15 function jumlahkanRumus(n) {
16     return (n * (n + 1)) / 2;
17 }
18
19 function cariPasangan(arr) {
20     const pasangan = [];
21     for (let i = 0; i < arr.length; i++) {
22         for (let j = i + 1; j < arr.length; j++) {
23             if (arr[i] + arr[j] === 10) pasangan.push([arr[i], arr[j]]);
24         }
25     }
26     return pasangan;
27 }
28
29 const data = Array.from({ length: 5000 }, (_, i) => i);
30 console.log("=== Perbandingan Waktu Eksekusi ===");
31 ukurWaktu("O(1) jumlahkanRumus ", () => jumlahkanRumus(N));
32 ukurWaktu("O(n) jumlahkanLinear ", () => jumlahkanLinear(N));
33 ukurWaktu("O(n2) cariPasangan ", () => cariPasangan(data));
34 console.log("\nHasil sama?", jumlahkanLinear(100) === jumlahkanRumus(100));
```

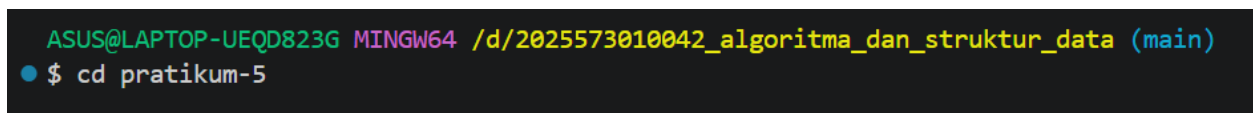
Nah untuk melihat hasilnya kalian pergi ke terminal dengan klik titi tiga kemudian pilih terminal, new terminal.



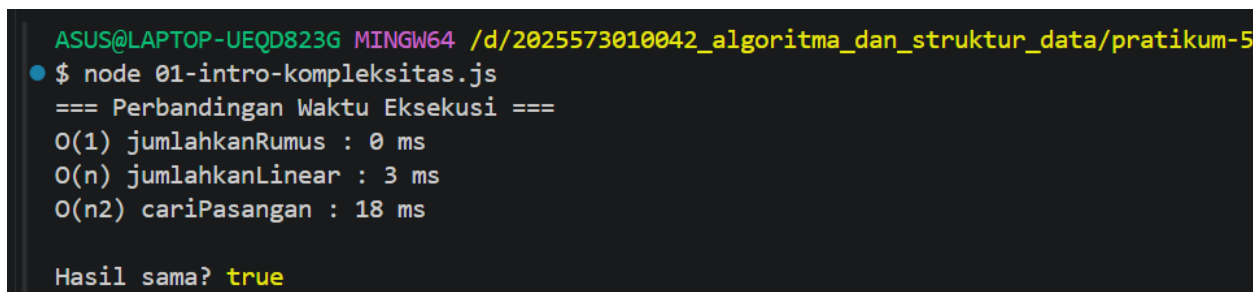
Setelah masuk ke halaman terminal seperti gambar dibawah kalian pilih untuk masuk ke halaman Git Bash. untuk pemahaman lebih jelas kalian bisa lihat urutan pada gambarnya.



Kemudian kalian ketikkan `cd pratikum-5` sesuai nama folder nya untuk memberi tahu terminal bahwa file yang ingin kita jalankan ada di dalam pratikum-5.



Kemudian ketikkan `node 01-intro-kompleksitas.js` untuk melihat hasilnya.



Langkah 5 | Buat file baru dengan nama 02-kelas-big-o.js

Kemudian di dalam halaman file **02-kelas-big-o.js** ketikkan kode dibawah ini :

```
pratikum-5 > JS 02-kelas-big-o.js > ...
1 // 02-kelas-big-o.js
2 // Contoh konkret setiap kelas kompleksitas Big O
3 // — O(1) – Konstan —
4 function ambilPertama(arr) {
5   return arr[0];
6 } // O(1)

7 function tambahkanItem(arr, item) {
8   arr.push(item);
9 } // O(1)
10 function isGenap(n) {
11   return n % 2 === 0;
12 } // O(1)

13 // — O(log n) – Logaritmik —
14 function binarySearch(arr, target) {
15   let kiri = 0,
16   kanan = arr.length - 1;
17   let langkah = 0;
18   while (kiri <= kanan) {
19     langkah++;
20     const tengah = Math.floor((kiri + kanan) / 2);
21     if (arr[tengah] === target) {
22       console.log(` Ditemukan di indeks ${tengah} setelah ${langkah} langkah`);
23       return tengah;
24     }
25     if (arr[tengah] < target) kiri = tengah + 1;
26     else kanan = tengah - 1;
27   }
28   return -1;
29 }

30 // — O(n) – Linear —
31 function cariMax(arr) {
32   let maks = arr[0];
33   for (const x of arr) if (x > maks) maks = x;
34   return maks;
35 }
```

```

36 // — O(n2) – Kuadratik —
37 function bubbleSortNaif(arr) {
38   const a = [...arr];
39   for (let i = 0; i < a.length; i++)
40     for (let j = 0; j < a.length - i - 1; j++)
41       if (a[j] > a[j + 1]) [a[j], a[j + 1]] = [a[j + 1], a[j]];
42   return a;
43 }

44 // — O(2n) – Eksponensial —
45 function fibRekursif(n) {
46   if (n <= 1) return n;
47   return fibRekursif(n - 1) + fibRekursif(n - 2); // dua panggilan rekursif!
48 }

49 // — Demonstrasi —
50 console.log("=== O(1) – selalu cepat ===");
51 console.log(ambilPertama([10, 20, 30, 40, 50])); // 10
52 console.log(isGenap(42)); // true
53 console.log("\n=== O(log n) – Binary Search ===");
54 const sorted = Array.from({ length: 1_000_000 }, (_, i) => i);
55 binarySearch(sorted, 731_452); // cari di 1 juta data
56 console.log("\n=== O(n) – Linear Search ===");
57 console.log(
58   "Max dari 1000 elemen:",
59   cariMax(Array.from({ length: 1000 }, () => (Math.random() * 1000) | 0)),
60 );

61 console.log("\n=== O(n2) – Bubble Sort (kecil saja) ===");
62 console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));
63 console.log("\n=== O(2n) – Fibonacci Rekursif (n kecil!) ===");
64 for (let i = 0; i <= 10; i++) process.stdout.write(fibRekursif(i) + " ");
65 console.log("");

```

```

66 // Demonstrasi betapa lambatnya O(2n)
67 console.log("\nWaktu fib(35) O(2n):");
68 let t = Date.now();
69 fibRekursif(35);
70 console.log(Date.now() - t, "ms");
71 console.log("Waktu fib(35) O(n) memoization:");
72 const memo = {};

73 function fibMemo(n) {
74     if (n <= 1) return n;
75     if (memo[n]) return memo[n];
76     return (memo[n] = fibMemo(n - 1) + fibMemo(n - 2));
77 }
78 t = Date.now();
79 fibMemo(35);
80 console.log(Date.now() - t, "ms");

```

Nah untuk melihat hasilnya kita lakukan lagi seperti langkah diatas membuka terminal GitBash kemudian ketik kode dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
• $ node 02-kelas-big-o.js

```

```

=== O(1) - selalu cepat ===
10
true

=== O(log n) - Binary Search ===
Ditemukan di indeks 731452 setelah 19 langkah

=== O(n) - Linear Search ===
Max dari 1000 elemen: 999

=== O(n2) - Bubble Sort (kecil saja) ===
[
  11, 12, 22, 25,
  34, 64, 90
]

```

```
=== O(2n) – Fibonacci Rekursif (n kecil!) ===  
0 1 1 2 3 5 8 13 21 34 55
```

Waktu fib(35) O(2n):

107 ms

Waktu fib(35) O(n) memoization:

0 ms

Langkah 6 | Buat file baru dengan nama 03-space-complexity.js

Kemudian di dalam halaman file **03-space-complexity.js** ketikkan kode dibawah ini :

```
pratikum-5 > JS 03-space-complexity.js > ...  
1  function jumlahArray(arr) {  
2      let total = 0; // hanya 1 variabel tambahan  
3      for (const x of arr) total += x;  
4      return total;  
5  }  
  
7  function duplikasiArray(arr) {  
8      const baru = []; // array baru tumbuh seiring arr  
9      for (const x of arr) baru.push(x * 2);  
10     return baru;  
11 }  
  
13 function faktorialRekursif(n) {  
14     if (n <= 1) return 1;  
15     return n * faktorialRekursif(n - 1); // n frame di call stack  
16 }  
17  
18 function faktorialIteratif(n) {  
19     let hasil = 1;  
20     for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel  
21     return hasil;  
22 }
```

```

24  const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
25
26  console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
27  console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
28  console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
29  console.log("Faktorial 10 iteratif :", faktorialIteratif(10));
30
31  function hitungUnik(arr) {
32      const seen = new Set(); // Set tumbuh hingga n elemen
33      for (const x of arr) seen.add(x);
34      return seen.size;
35  }
36
37  const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
38  console.log("Elemen unik:", hitungUnik(dataAcak)); // 7
39

```

Untuk melihat hasil nya masih mengikuti tahap di atas kemudian ketikkan kode dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
● $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
● $ node 03-space-complexity.js

Jumlah array : 55
Duplikasi array: [
    2,  4,  6,  8, 10,
    12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 7

```

Kemudian dibawah file `03-space-complexity.js` kalian ketikkan kode `latihan-2` seperti yang di arahkan pada modul.

```
40 console.log("");
41 console.log("=== latihan2 ===");
42
50 function cariPasanganLambat(arr, target) {
51   for (let i = 0; i < arr.length; i++) {
52     for (let j = i + 1; j < arr.length; j++) {
53       if (arr[i] + arr[j] === target) {
54         return [arr[i], arr[j]];
55       }
56     }
57   }
58   return null;
59 }
60
64 function cariPasanganCepat(arr, target) {
65   const angkaDilihat = new Set();
66   for (let angka of arr) {
67     let selisih = target - angka;
68     if (angkaDilihat.has(selisih)) {
69       return [selisih, angka];
70     }
71     angkaDilihat.add(angka);
72   }
73   return null;
74 }
75
77 const testArray = [2, 7, 11, 15];
78 const targetUji = 9;
79
80 console.log("--- Uji Coba Sederhana ---");
81 console.log("Hasil Lambat:", cariPasanganLambat(testArray, targetUji)); // [2, 7]
82 console.log("Hasil Cepat:", cariPasanganCepat(testArray, targetUji)); // [2, 7]
83
85 console.log("\n--- Pengukuran Waktu (50.000 Data) ---");
86
87 const dataBesar = Array.from({ length: 50000 }, () =>
88   Math.floor(Math.random() * 100000),
89 );
90 const targetTidakAda = -1; // Target yang pasti tidak ada agar loop berjalan maksimal
```

```

93   const mulaiLambat = Date.now();
94   cariPasanganLambat(dataBesar, targetTidakAda);
95   const selesaiLambat = Date.now();
96   console.log(`Waktu cariPasanganLambat: ${selesaiLambat - mulaiLambat}ms`);
97
99   const mulaiCepat = Date.now();
100  cariPasanganCepat(dataBesar, targetTidakAda);
101  const selesaiCepat = Date.now();
102  console.log(`Waktu cariPasanganCepat: ${selesaiCepat - mulaiCepat}ms`);
103

```

Hasilnya :

```

=== latihan2 ===
--- Uji Coba Sederhana ---
Hasil Lambat: [ 2, 7 ]
Hasil Cepat: [ 2, 7 ]

--- Pengukuran Waktu (50.000 Data) ---
Waktu cariPasanganLambat: 1653ms
Waktu cariPasanganCepat: 10ms

```

Langkah 7 | Buat file baru dengan nama 04-refactor.js

Kemudian pada halaman file **04-refactor.js** ketikkan kode dibawah ini :

```

pratikum-5 > JS 04-refactor.js > ...
1   // 04-space-complexity.js
2   // --- O(1) Space: hanya pakai variabel tambahan konstan ---
3   function jumlahArray(arr) {
4       let total = 0; // hanya 1 variabel tambahan
5       for (const x of arr) total += x;
6       return total;
7   }

8   // --- O(n) Space: membuat struktur baru sebesar input ---
9   function duplikasiArray(arr) {
10      const baru = []; // array baru tumbuh seiring arr
11      for (const x of arr) baru.push(x * 2);
12      return baru;
13  }

```

```

14 // --- O(n) Space: rekursi (call stack) ---
15 function faktorialRekursif(n) {
16     if (n <= 1) return 1;
17     return n * faktorialRekursif(n - 1); // n frame di call stack
18 }

19 // --- O(1) Space: faktorial iteratif ---
20 function faktorialIteratif(n) {
21     let hasil = 1;
22     for (let i = 2; i <= n; i++) hasil *= i; // hanya 2 variabel
23     return hasil;
24 }

25 const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
26 console.log("Jumlah array :", jumlahArray(arr)); // O(1) space
27 console.log("Duplikasi array:", duplikasiArray(arr)); // O(n) space
28 console.log("Faktorial 10 rekursif :", faktorialRekursif(10));
29 console.log("Faktorial 10 iteratif :", faktorialIteratif(10));

30 // Visualisasi: hitung elemen unik (O(n) space)
31 function hitungUnik(arr) {
32     const seen = new Set(); // Set tumbuh hingga n elemenfor (const x of arr) seen.add(x);
33     return seen.size;
34 }
35 const dataAcak = [1, 2, 3, 2, 1, 4, 5, 3, 6, 4, 7];
36 console.log("Elemen unik:", hitungUnik(dataAcak)); // 7

```

Untuk melihat hasilnya ketikkan kode dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
• $ node 04-refactor.js

```

```

Jumlah array : 55
Duplikasi array: [
    2,  4,  6,  8, 10,
   12, 14, 16, 18, 20
]
Faktorial 10 rekursif : 3628800
Faktorial 10 iteratif : 3628800
Elemen unik: 0

```

2.2. Latihan dan Tugas Pratikum

Latihan - 1 | Buat file baru dengan nama latihan-1.js.

Kemudian pada halaman file **Latihan-1.js** ketikkan kode dibawah ini :

```
pratikum-5 > JS latihan-1.js > ...
1  /**
2   * Latihan 1: Identifikasi Kompleksitas
3   */
4
5  // 8. Helper function untuk menghitung waktu eksekusi
6  function hitungKompleksitas(n, fn, inputTambahan = null) {
7      const mulai = Date.now();
8      fn(n, inputTambahan);
9      const selesai = Date.now();
10     console.log(`Waktu eksekusi: ${selesai - mulai}ms`);
11 }

16 // Big O: O(1) - Constant Time
17 // Alasan: Operasi perkalian dilakukan tepat satu kali, tidak peduli seberapa besar nilai n.
18 function fungsiA(n) {
19     return n * 2;
20 }

23 // Big O: O(n^2) - Quadratic Time
24 // Alasan: Terdapat nested loop (loop di dalam loop). Loop luar berjalan n kali
25 // dan loop dalam juga berjalan n kali, sehingga total operasinya adalah n * n.
26 function fungsiB(n) {
27     for (let i = 0; i < n; i++) {
28         for (let j = 0; j < n; j++) {
29             // console.log(i, j); // Dimatikan agar tidak memenuhi terminal
30         }
31     }
32 }

35 // Big O: O(log n) - Logarithmic Time
36 // Alasan: Variabel i dilipatgandakan (i *= 2) pada setiap iterasi.
37 // Jumlah langkah yang dibutuhkan untuk mencapai n tumbuh secara logaritmik.
38 function fungsiC(n) {
39     for (let i = 1; i < n; i *= 2) {
40         // console.log(i);
41     }
42 }
```

```

45 // Big O:  $O(n^3)$  - Cubic Time
46 // Alasan: Terdapat tiga lapis perulangan (forEach di dalam forEach di dalam forEach).
47 // Jika panjang array adalah n, maka total operasinya adalah  $n * n * n$ .
48 function fungsiD(n, arr) {
49     arr.forEach((x) => {
50         arr.forEach((y) => {
51             arr.forEach((z) => {
52                 // console.log(x, y, z);
53             });
54         });
55     });
56 }

60 const n = 1000;
61 const dummyArray = new Array(n).fill(0); // Digunakan untuk Fungsi D
62
63 console.log("Menguji Fungsi A ( $O(1)$ ):");
64 hitungKompleksitas(n, fungsiA);

66 console.log("\nMenguji Fungsi B ( $O(n^2)$ ):");
67 hitungKompleksitas(n, fungsiB);
68
69 console.log("\nMenguji Fungsi C ( $O(\log n)$ ):");
70 hitungKompleksitas(n, fungsiC);

72 console.log("\nMenguji Fungsi D ( $O(n^3)$ ):");
73 // Catatan:  $O(n^3)$  dengan  $n=1000$  berarti 1 miliar operasi.
74 // Ini mungkin memakan waktu beberapa detik tergantung spek komputer Anda.
75 hitungKompleksitas(n, () => fungsiD(n, dummyArray));
76

```

untuk melihat hasilnya ketikkan kode dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
● $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
● $ node latihan-1.js

Menguji Fungsi A ( $O(1)$ ):
Waktu eksekusi: 0ms

Menguji Fungsi B ( $O(n^2)$ ):
Waktu eksekusi: 18ms

Menguji Fungsi C ( $O(\log n)$ ):
Waktu eksekusi: 0ms

Menguji Fungsi D ( $O(n^3)$ ):
Waktu eksekusi: 604ms

```

Tugas

Buat folder baru dengan nama tugas, kemudian buat dua file baru dengan nama tugas-1.js dan tugas-2.js.

- Pada halaman file **tugas-1.js** ketikkan kode dibawah ini :

```
pratikum-5 > tugas > JS tugas-1.js > intersectionSlow
1  /**
2   * Tugas 1: Analisis dan Refactor
3   */
4
5  // Helper untuk mengukur waktu
6  function measure(label, fn) {
7      const start = Date.now();
8      const result = fn();
9      const end = Date.now();
10     console.log(`${label}: ${end - start}ms`);
11     return result;
12 }

18 // Versi O(n^2) - Time: O(n*m), Space: O(min(n,m))
19 function intersectionSlow(arr1, arr2) {
20     return arr1.filter((item) => arr2.includes(item));
21 }

23 // Versi O(n) - Time: O(n+m), Space: O(n)
24 function intersectionFast(arr1, arr2) {
25     const set1 = new Set(arr1);
26     return arr2.filter((item) => set1.has(item));
27 }

28

33 // Time: O(n * k log k) di mana n adalah jumlah kata, k panjang kata terpanjang
34 // Space: O(n * k)
35 function groupAnagrams(strs) {
36     const map = {};
37     for (let s of strs) {
38         // Sortir kata untuk dijadikan key (contoh: 'eat' -> 'aet')
39         const key = s.split("").sort().join("");
40         if (!map[key]) map[key] = [];
41         map[key].push(s);
42     }
43     return Object.values(map);
44 }
```

```

50 // Versi  $O(n^3)$  - Triple Nested Loop
51 function isSquareSumSlow(arr) {
52   for (let i = 0; i < arr.length; i++) {
53     for (let j = i + 1; j < arr.length; j++) {
54       for (let k = 0; k < arr.length; k++) {
55         if (arr[i] + arr[j] === Math.pow(arr[k], 2)) return true;
56       }
57     }
58   }
59   return false;
60 }

62 // Versi  $O(n \log n)$  - Sorting + Two Pointers
63 function isSquareSumFast(arr) {
64   arr.sort((a, b) => a - b); //  $O(n \log n)$ 
65   const squares = arr.map((x) => x * x); // Simpan kuadrat agar hemat hitung
66
67   for (let target of squares) {
68     let left = 0;
69     let right = arr.length - 1;
70     while (left < right) {
71       const sum = arr[left] + arr[right];
72       if (sum === target) return true;
73       if (sum < target) left++;
74       else right--;
75     }
76   }
77   return false;
78 }

84 const bigData1 = Array.from({ length: 30000 }, (_, i) => i);
85 const bigData2 = Array.from({ length: 30000 }, (_, i) => i + 15000);
86
87 console.log("--- Uji Fungsi A (Intersection) ---");
88 measure("Intersection  $O(n^2)$ ", () => intersectionSlow(bigData1, bigData2));
89 measure("Intersection  $O(n)$ ", () => intersectionFast(bigData1, bigData2));
90
91 console.log("\n--- Uji Fungsi B (Anagram) ---");
92 const words = ["eat", "tea", "tan", "ate", "nat", "bat"];
93 console.log("Hasil:", JSON.stringify(groupAnagrams(words)));
94
95 console.log("\n--- Uji Fungsi C (Two Sum Square) ---");
96 const bigNumbers = Array.from({ length: 2000 }, () =>
97   Math.floor(Math.random() * 1000),
98 );
99 //  $O(n^3)$  dengan 2000 data mungkin sangat lama, kita coba ukur
100 measure("Square Sum  $O(n^3)$ ", () => isSquareSumSlow(bigNumbers));
101 measure("Square Sum  $O(n \log n)$ ", () => isSquareSumFast(bigNumbers));
102

```

Untuk melihat hasilnya kalian ketikkan kode dibawah ini :

```
ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
• $ node tugas/tugas-1.js

--- Uji Fungsi A (Intersection) ---
Intersection  $O(n^2)$ : 476ms
Intersection  $O(n)$ : 5ms

--- Uji Fungsi B (Anagram) ---
Hasil: [["eat","tea","ate"],["tan","nat"],["bat"]]

--- Uji Fungsi C (Two Sum Square) ---
Square Sum  $O(n^3)$ : 1ms
Square Sum  $O(n \log n)$ : 1ms
```

- Pada halaman file **tugas-2.js** ketikkan kode dibawah ini :

```
pratikum-5 > tugas > JS tugas-2.js > ...
1  /**
2   * Tugas 2: Visualisasi Pertumbuhan Big O
3   */
4
5  // 3. fn_O1(n) -> Constant Time  $O(1)$ 
6  function fn_O1(n) {
7      return n + 1;
8  }

10 // 4. fn_On(n) -> Linear Time  $O(n)$ 
11 function fn_On(n) {
12     let sum = 0;
13     for (let i = 0; i < n; i++) {
14         sum += i;
15     }
16     return sum;
17 }
```

```

19 // 5. fn_OnLogn(n) -> Linearithmic Time  $O(n \log n)$ 
20 function fn_OnLogn(n) {
21     let count = 0;
22     for (let i = 0; i < n; i++) {
23         // Loop dalam berjalan  $\log_2(n)$  kali
24         for (let j = 1; j < n; j *= 2) {
25             count++;
26         }
27     }
28     return count;
29 }

```

```

31 // 6. fn_On2(n) -> Quadratic Time  $O(n^2)$ 
32 function fn_On2(n) {
33     let count = 0;
34     for (let i = 0; i < n; i++) {
35         for (let j = 0; j < n; j++) {
36             count++;
37         }
38     }
39     return count;
40 }

```

```

42 // 2. Function benchmarkSemua
43 function benchmarkSemua(ukuranData) {
44     console.log("Ukuran (n) |  $O(1)$  |  $O(n)$  |  $O(n \log n)$  |  $O(n^2)$ ");
45     console.log("-----");
46
47     ukuranData.forEach((n) => {
48         // Helper untuk mengukur waktu dalam mikrosekond (lebih presisi dari ms)
49         const hitungWaktu = (fn) => {
50             const start = process.hrtime.bigint();
51             fn(n);
52             const end = process.hrtime.bigint();
53             // Kembalikan dalam mikrosekond ( $\mu s$ )
54             return Number(end - start) / 1000;
55         };

```

```

57         const t1 = hitungWaktu(fn_O1);
58         const tn = hitungWaktu(fn_On);
59         const tnLog = hitungWaktu(fn_OnLogn);
60         const tn2 = hitungWaktu(fn_On2);

```

```

62     console.log(
63         `${n.toString().padEnd(10)} | ` +
64         `${t1.toFixed(2).padEnd(7)} | ` +
65         `${tn.toFixed(2).padEnd(7)} | ` +
66         `${tnLog.toFixed(2).padEnd(10)} | ` +
67         `${tn2.toFixed(2)} μs`,
68     );
69 });
70 }

```

```

73     benchmarkSemua([100, 500, 1000, 5000, 10000]);
74

```

Untuk melihat hasilnya ketikkan kode dibawah ini :

```

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data (main)
• $ cd pratikum-5

ASUS@LAPTOP-UEQD823G MINGW64 /d/2025573010042_algoritma_dan_struktur_data/pratikum-5 (main)
• $ node tugas/tugas-2.js

```

Ukuran (n)	$O(1)$	$O(n)$	$O(n \log n)$	$O(n^2)$
100	7.60	15.90	22.20	208.60 μs
500	18.60	15.70	98.60	1267.40 μs
1000	9.90	31.00	31.30	802.80 μs
5000	0.80	10.00	117.50	49368.60 μs
10000	0.90	21.20	316.50	168216.30 μs

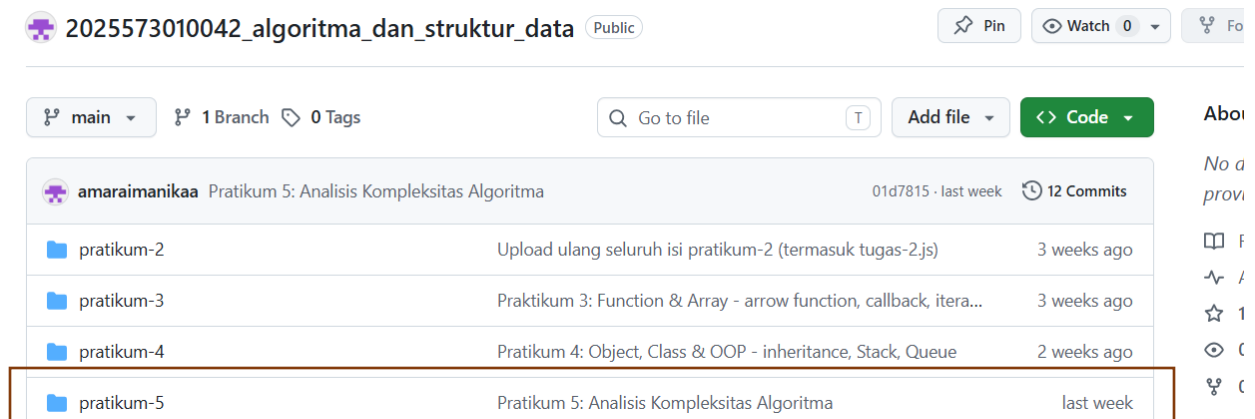
2.3. Melakukan Commit dan Push

Langkah 8 | Lakukan commit dan push ke GitHub.

Langkah awal kalian buka terminal GitBash kemudian kalian ketikkan kode dibawah ini untuk melakukan commit dan push ke GitHub.

1. `cd ..`
2. `git status`
3. `git add pratikum-5/`
4. `git commit -m "Pratikum 5: Analisis Kompleksitas Algoritma"`
5. `git push origin main`

jika berhasil maka di GitHub kalian akan muncul seperti gambar dibawah ini :



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan analisis yang dilakukan pada Modul 5, dapat disimpulkan bahwa:

1. Analisis kompleksitas algoritma memberikan ukuran objektif untuk menilai efisiensi algoritma tanpa bergantung pada perangkat keras yang digunakan.
2. Notasi **Big-O** sangat efektif dalam menentukan skenario terburuk (*worst-case*) dari sebuah algoritma, yang menjadi standar utama dalam pengembangan perangkat lunak.
3. Semakin tinggi tingkat kompleksitas sebuah algoritma (misalnya dari $O(n)$ ke $O(n^2)$), maka penurunan performa akan sangat terasa seiring dengan bertambahnya jumlah data masukan.

3.2 Saran

1. Dalam menulis kode, mahasiswa disarankan untuk menghindari penggunaan *nested loop* (perulangan bersarang) yang tidak perlu, karena hal ini dapat meningkatkan kompleksitas waktu secara signifikan menjadi kuadratik ($O(n^2)$).
2. Perlu dilakukan pengujian secara langsung menggunakan fungsi waktu di berbagai bahasa pemrograman untuk membandingkan hasil perhitungan teoritis dengan waktu eksekusi riil.
3. Sebaiknya mahasiswa juga mempelajari analisis *Average Case* dan *Best Case* agar memiliki pemahaman yang komprehensif terhadap perilaku algoritma di berbagai kondisi.